Folio

A course assistant inside IntelliJ.
It answers from the lectures, and it shows the page.

罗臻 Luo Zhen
2451989

李灿文 Li Canwen
2452270

范羽 Fan Yu
2452750

孟圣雨 Meng Shengyu
2452272

02 PROJECT

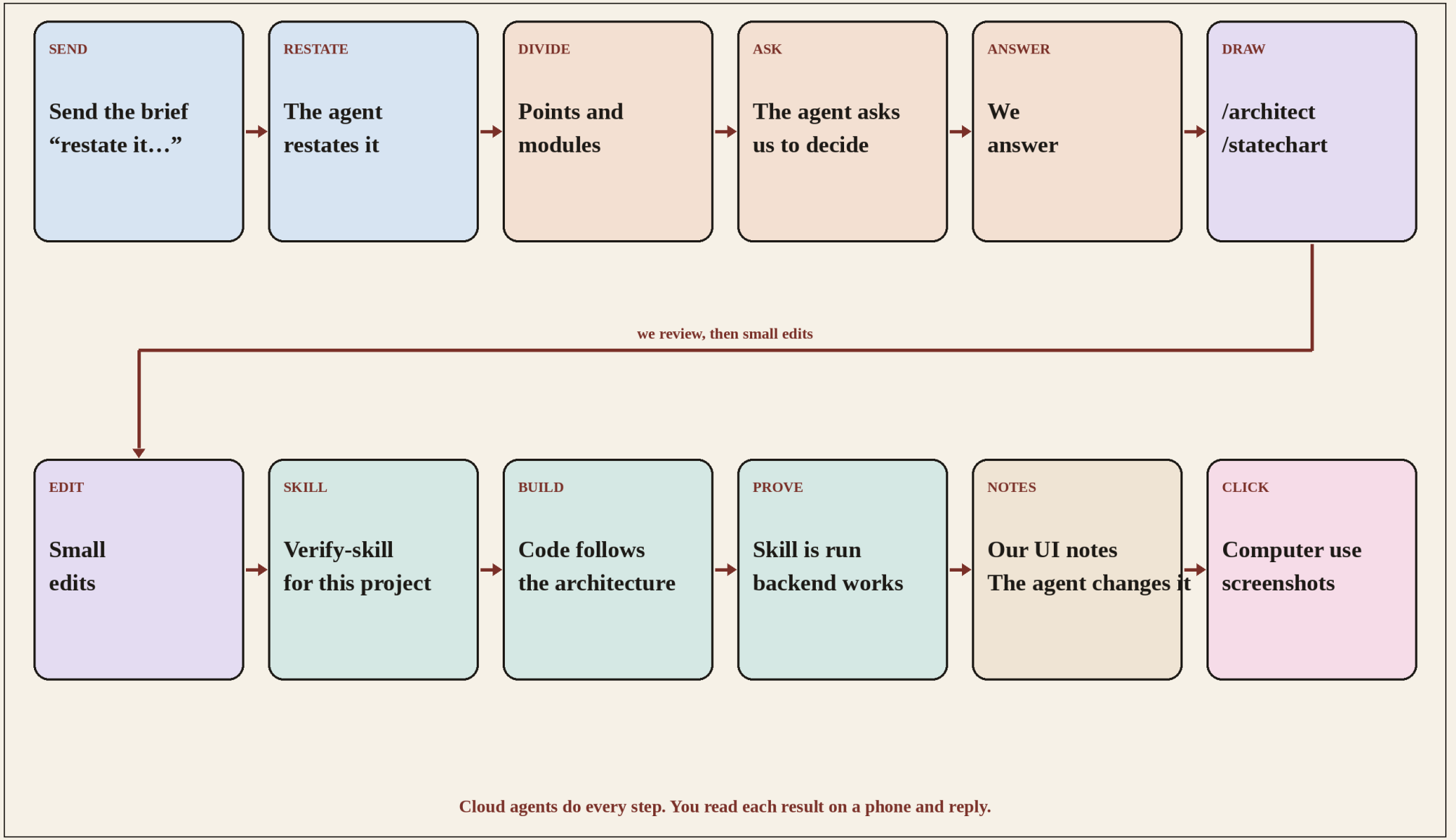
What it is

It is a tool window on the right of IntelliJ.
You import the lectures once. They stay here.
You type a question, or you select code
and ask about that code.
The answer is written from the pages that match.
It names the file and the page.
In the picture, the question asks which
annotation a JUnit 5 test method uses.
The answer is @Test, from Lec-3.4,
pages 3 and 2. The titles open those pages.
F0 is that cited answer. The page has to be
a real page. If the lectures do not cover it,
you still get an answer, and no page is invented.
F1 is three further actions. Practice gives
three exercises. Tests show a JUnit file first,
and write it only when you click. Commit text
is a message from the staged files. You copy it.
Folio does not commit.
Group C rewrites the student's code and drives
the diff viewer. This project does not.



How it was made

03 MAKING



Decisions that stayed with us

After the restatement, the agent listed choices it would not make alone. These are the answers we gave.

Retrieval

We index the lectures on this machine, by the words in them.
We do not send the files out to be turned into vectors.
The page is still there when the network is down,
because the index never left the machine.

The third action

The brief asks for three actions besides answering.
Answering again would only repeat that.
Rewriting the student’s code is what Group C does.
We use this action for practice: three exercises,
and an explanation you can open. The editor is not changed.

Tests

The model may draft a JUnit test. It may not save the file.
You see the Java first. The file appears only after Write,
and you can undo it.

Commit messages

The model may draft the message from the staged files.
It may not run git commit. You copy the message,
or you put it into the commit box yourself.

The model

The address and the model name live in settings.
This demonstration uses DeepSeek. The key is in the
operating system keyring, not in the project.

The library, and a question

04 THE WINDOW

You bring the lectures in from the course-files button.

Folio accepts PDF, PPTX, TXT, Markdown, and DOCX.

It reads them on this machine. Indexing needs no key.

The library is shared by every project in this IDE.

Removing a file asks first. The original stays on disk.

You ask in the box, or about selected code.

The answer arrives as it is written.

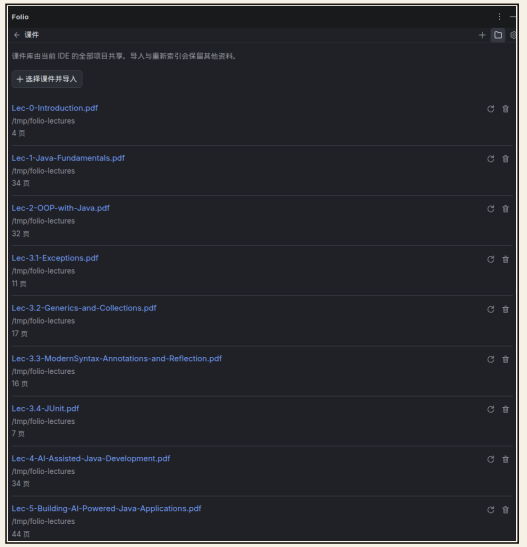
Thinking stays in its own block.

If you stop, the words already shown remain.

A used page is marked [1]. The title opens it.

A miss is still an ordinary answer. No invented page.

With no key, search runs and no model call is made.





Practice, settings, and drafts

04 THE WINDOW

Practice gives three exercises, from a topic or from selected code.

They appear together only after the reply parses as three exercises.

Until then you see a status line, not a stream of tokens.

You open one explanation at a time. The editor is not changed.

Settings holds the address and the model.

The key is in the operating system keyring.

After it is saved, the field is empty.

The line under it says a key for this address is saved.

Indexing never needs that key. Asking the model does.

Ask, practice, tests, and commit text each keep a draft.

Changing mode does not throw a draft away.

A new chat clears the transcript. The drafts stay.

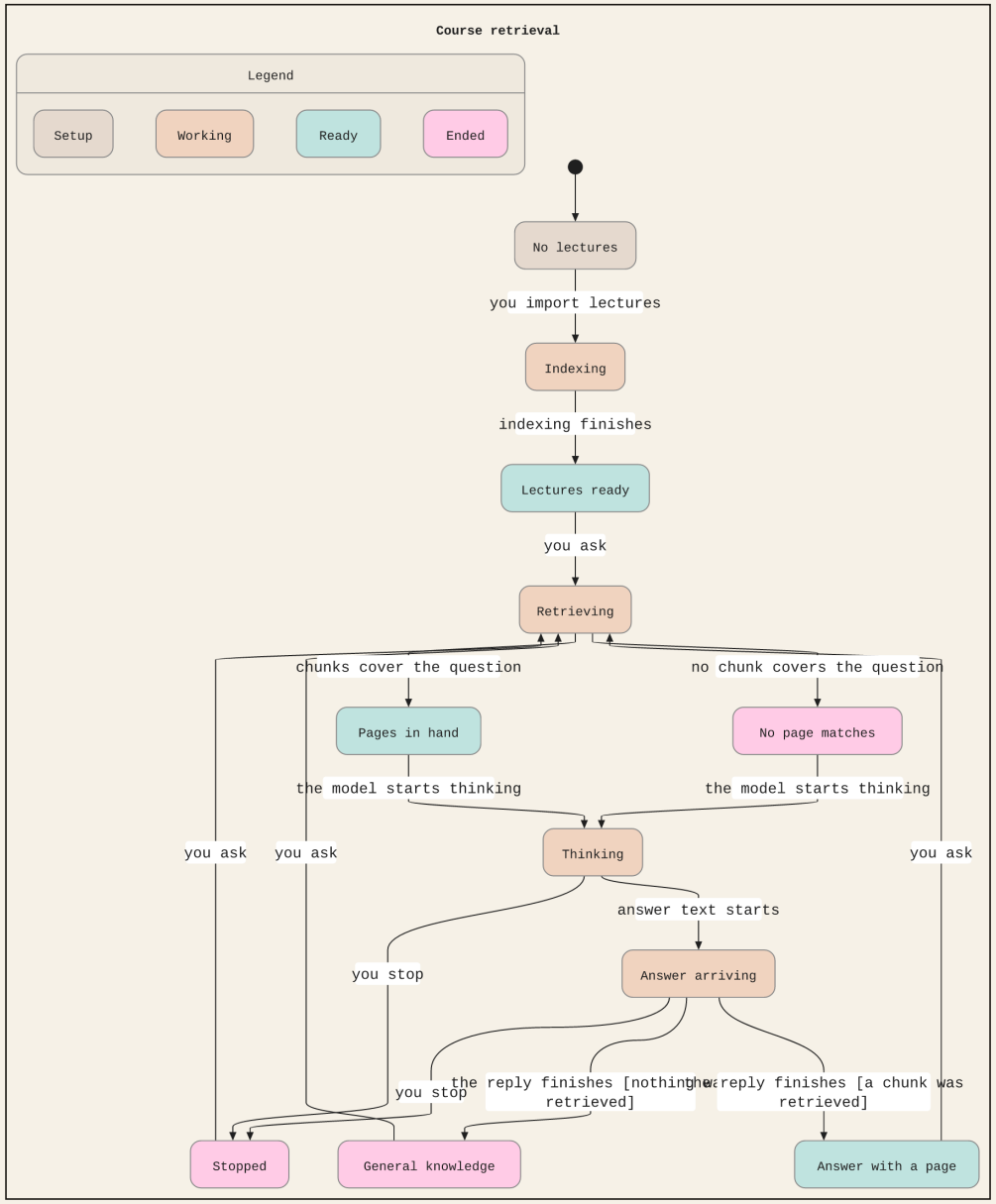




How a file becomes a page

05 HOW IT WORKS

Folio does not invent page numbers.
A PDF keeps its own pages, including blanks.
A slide deck keeps slide order.
A text file is a section, not a made-up page.
Each of those is cut at about 700 characters.
The next piece starts 100 characters earlier,
so a sentence on the cut is not lost.
The pieces are indexed here, by their words.
There is no embedding model and no web search.
A question, plus any selected code, is matched.
Folio keeps at most four pieces.
A piece must cover at least half the terms.
One shared word in a longer question misses.
The model writes [1] for a piece it uses.
[1] becomes that file and page. The title opens it.
Stopping does not invent a page.
The drawing is the same rule.



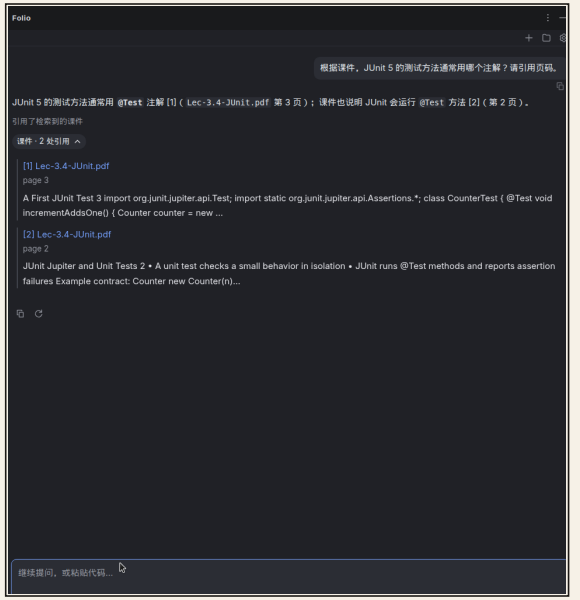
The question, the answer, and the page

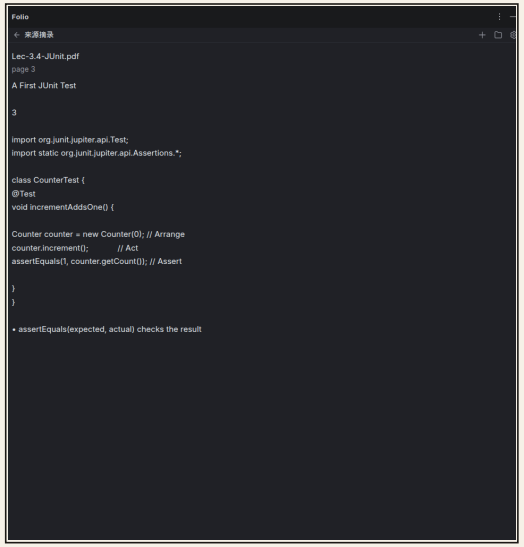
05 HOW IT WORKS

The question was:

根据课件，JUnit 5 的测试方法通常用哪个注解？

Folio searches the lectures on this machine
and keeps the pages that bear on the question.
Here those pages are Lec-3.4, pages 3 and 2.
They are the pages that introduce @Test.
The answer says the annotation is @Test.
It marks the two pages as [1] and [2].
The line under the answer is the file name
and the page number.
Choosing a title opens that page.
The lower picture is the page that opened.
The numbers are pages that were retrieved.
Folio does not invent a file or a page.





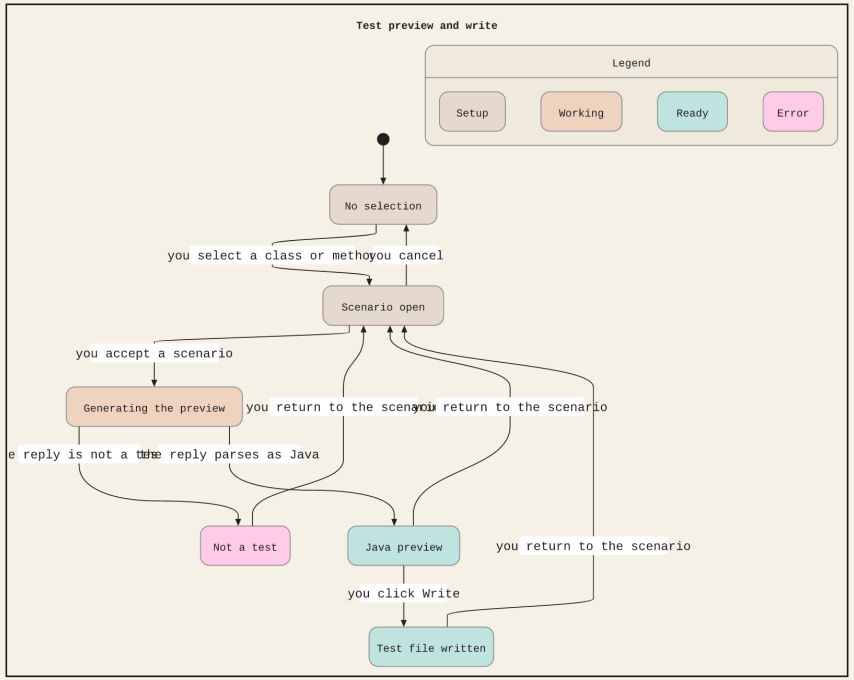
A test file, and a commit message

05 HOW IT WORKS

Asking and practice only put words on the screen. These two can add something to the project, and only after you click.

Writing a test file

You say what you want tested.
Folio shows the Java it would write.
It checks that the text is legal Java
and that it looks like a JUnit test.
It does not compile the test, and it does not run it.
Nothing is saved until you click Write.
Then the file is in the project, and you can undo it.
Cancel, or a preview that is not a test,
and no file is created.



Drafting a commit message

You press Stage yourself. That stages the changes.
Folio reads what is staged.
It shows three possible messages.
You copy one, or you put it into the commit box.
If you stage something else afterwards,
copy and fill stop working.
The message was written for the old changes.
Folio never runs git commit.
You commit yourself.

